

Souvenirs

Amaru kauft Souvenirs in einem ausländischen Geschäft. Es gibt N **Arten** von Souvenirs. Es gibt unendlich viele Souvenirs jeder Art in dem Geschäft.

Jede Art von Souvenir hat einen festen Preis. Ein Souvenir der Art i ($0 \leq i < N$) hat einen Preis von $P[i]$ Münzen, wobei $P[i]$ eine positive ganze Zahl ist.

Amaru weiß, dass die Souvenirarten in absteigender Reihenfolge nach Preis sortiert sind und die Preise der Souvenirs unterschiedlich sind. Genauer gesagt, $P[0] > P[1] > \dots > P[N-1] > 0$. Außerdem konnte er den Wert von $P[0]$ in Erfahrung bringen. Leider kennt Amaru keine weiteren Informationen über die Preise der Souvenirs.

Um einige Souvenirs zu kaufen, wird Amaru eine Reihe von Transaktionen mit dem Verkäufer durchführen.

Jede Transaktion besteht aus den folgenden Schritten:

1. Amaru gibt dem Verkäufer eine (positive) Anzahl von Münzen.
2. Der Verkäufer legt die Münzen auf einen Stapel auf den Tisch im Hinterzimmer, wo Amaru sie nicht sehen kann.
3. Der Verkäufer betrachtet jede Souvenirart in der Reihenfolge $0, 1, \dots, N-1$ eine nach der anderen. Jede Art wird in dieser Transaktion **genau einmal** betrachtet.
 - Wenn beim Betrachten der Souvenirart i , die aktuelle Anzahl der Münzen auf dem Stapel mindestens $P[i]$ beträgt, dann
 - nimmt der Verkäufer $P[i]$ Münzen vom Stapel, und
 - legt ein Souvenir der Art i auf den Tisch.
4. Der Verkäufer gibt Amaru alle verbliebenen Münzen vom Stapel und alle Souvenirs vom Tisch.

Beachte, dass keine Münzen oder Souvenirs auf dem Tisch liegen, bevor eine Transaktion beginnt.

Deine Aufgabe ist es, Amaru anzuweisen, eine bestimmte Anzahl von Transaktionen durchzuführen, sodass:

- er bei jeder Transaktion **mindestens ein** Souvenir kauft, und
- er insgesamt **exakt** i Souvenirs der Art i kauft, für jedes i ($0 \leq i < N$). Das bedeutet auch, dass Amaru kein Souvenir des Typs 0 kaufen sollte.

Amaru muss die Anzahl der Transaktionen nicht minimieren und verfügt über einen unbegrenzten Vorrat an Münzen.

Implementierungsdetails

Implementiere die folgende Funktion:

```
void buy_souvenirs(int N, long long P0)
```

- N : die Anzahl der Souvenirarten.
- $P0$: der Wert von $P[0]$.
- Diese Funktion wird für jeden Testfall genau einmal aufgerufen.

Diese Funktion kann die folgende Funktion aufrufen, um Amaru anzuweisen, eine Transaktion zu machen:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : die Anzahl der Münzen, die Amaru dem Verkäufer gegeben hat.
- Die Funktion gibt ein Paar zurück. Das erste Element des Paares ist ein Array L , das die Souvenirarten enthält, die gekauft wurden (in aufsteigender Reihenfolge). Das zweite Element ist eine ganze Zahl R , die Anzahl der Münzen, die nach der Transaktion an Amaru zurückgegeben wurden.
- Es ist erforderlich, dass $P[0] > M \geq P[N - 1]$. Die Bedingung $P[0] > M$ stellt sicher, dass Amaru kein Souvenir des Typs 0 kauft, und $M \geq P[N - 1]$ stellt sicher, dass Amaru mindestens ein Souvenir kauft. Wenn diese Bedingungen nicht erfüllt sind, erhält die Lösung die Bewertung `Output isn't correct: Invalid argument`. Beachte, dass im Gegensatz zu $P[0]$ der Wert von $P[N - 1]$ nicht in der Eingabe enthalten ist.
- Diese Funktion kann höchstens 5000 Mal in jedem Testfall aufgerufen werden.

Das Verhalten des Graders ist **nicht adaptiv**. Das bedeutet, dass die Reihenfolge der Preise P feststeht, bevor `buy_souvenirs` aufgerufen wird.

Einschränkungen

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ für alle i ($0 \leq i < N$).
- $P[i] > P[i + 1]$ für alle i ($0 \leq i < N - 1$).

Subtasks

Subtask	Punkte	Weitere Einschränkungen
1	4	$N = 2$
2	3	$P[i] = N - i$ für alle i ($0 \leq i < N$).
3	14	$P[i] \leq P[i + 1] + 2$ für alle i ($0 \leq i < N - 1$).
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ für alle i ($0 \leq i < N - 2$). $P[i] \leq 2 \cdot P[i + 1]$ für alle i ($0 \leq i < N - 1$).
6	33	Keine weiteren Einschränkungen.

Beispiel

Betrachte den folgenden Funktionsaufruf.

```
buy_souvenirs(3, 4)
```

Es gibt $N = 3$ Arten von Souvenirs und $P[0] = 4$. In diesem Fall, gibt es nur drei mögliche Preisfolgen P : $[4, 3, 2]$, $[4, 3, 1]$, und $[4, 2, 1]$.

Nehmen wir an, dass `buy_souvenirs` den folgenden Aufruf macht: `transaction(2)`. Weiter nehmen wir an, dass der Aufruf `([2], 1)` zurückgibt. Das heißt, dass Amaru ein Souvenir der Art 2 gekauft hat und dass der Verkäufer ihm 1 Münze zurückgegeben hat. Daraus lässt sich ableiten, dass $P = [4, 3, 1]$ ist, da:

- Für $P = [4, 3, 2]$, die `transaction(2)` das Paar `([2], 0)` zurückgegeben hätte.
- Für $P = [4, 2, 1]$, die `transaction(2)` das Paar `([1], 0)` zurückgegeben hätte.

Nun kann `buy_souvenirs` den Aufruf `transaction(3)` machen, welcher das Paar `([1], 0)` zurückgibt, was bedeutet, dass Amaru ein Souvenir der Art 1 gekauft hat und der Verkäufer ihm 0 Münzen zurückgegeben hat. Bislang hat er insgesamt ein Souvenir der Art 1 und ein Souvenir der Art 2 gekauft.

Schlussendlich kann `buy_souvenirs` den Aufruf `transaction(1)` machen, welcher `([2], 0)` zurückgibt, was bedeutet, dass Amaru ein Souvenir der Art 2 gekauft hat. Beachte, dass wir hier auch `transaction(2)` hätten benutzen können. Jetzt hat Amaru also, insgesamt ein Souvenir der Art 1 und zwei Souvenirs der Art 2 gekauft, wie erforderlich.

Beispielgrader

Eingabeformat:

```
N  
P[0] P[1] ... P[N-1]
```

Ausgabeformat:

```
Q[0] Q[1] ... Q[N-1]
```

Dabei ist $Q[i]$ die Anzahl der gekauften Souvenirs der Art i für alle i ($0 \leq i < N$).